

Pricing Lab

Most derivatives have no formula, so banks price them by simulation. This chapter covers the four problems every Monte Carlo pricer has to solve: making estimates precise, computing sensitivities (the Greeks), handling path-dependent payoffs, and pricing the right to exercise early.

9.1 Monte Carlo & Variance Reduction

A price is a discounted average of payoffs over risk-neutral paths (section 6.2), so simulate many paths and average. The catch is precision: the error shrinks only like $1/\sqrt{n}$, so 100× more paths buy just 10× more accuracy. *Variance reduction* gets the same accuracy from far fewer paths, by averaging something with the same mean but less noise.

Predict first: For a deep out-of-the-money call (strike 150, stock at 100), which method will give the smallest error for the same number of payoff evaluations?

Antithetic variates

Control variate

Importance sampling

Answer: "Importance sampling". Almost every path of a deep out-of-the-money call pays nothing, so plain Monte Carlo wastes them. Importance sampling moves the paths to where the payoff happens and reweights them, cutting the standard error about eightfold at $K = 150$.

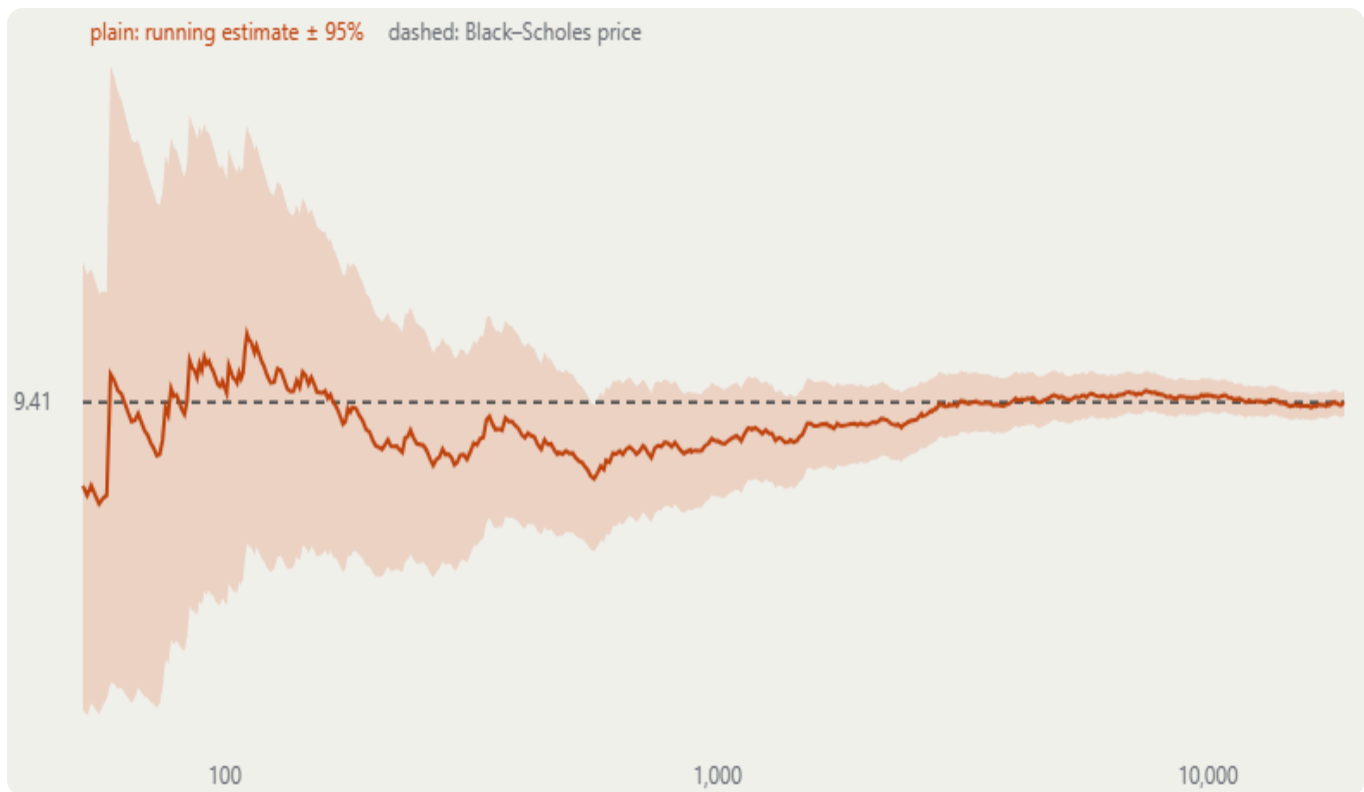
All four methods use the same 20,000 random numbers. Pick a method and a strike. The band is the 95% confidence interval as more payoffs are averaged, and the dashed line is the exact Black–Scholes price.

$$\hat{C} = e^{-rT} \frac{1}{n} \sum_{i=1}^n (S_T^{(i)} - K)^+, \quad \text{standard error} = \frac{\text{sd}}{\sqrt{n}}, \quad \hat{C}_{\text{cv}} = \hat{C} - \beta(\bar{S}_T - S_0)$$



$S_T^{(i)}$	The stock price at maturity on simulated path i , drawn from the risk-neutral GBM (growth rate $r = 3\%$, volatility 20%).
$(S_T - K)^+$	The call's payoff: $S_T - K$ if positive, otherwise 0.
e^{-rT}	Discounting: a payoff in one year is worth less today.
sd, standard error	sd is the spread of individual payoffs. The standard error, $\text{sd}/\sqrt{n}$, is the typical error of their average.
antithetic	Use each random number Z twice, as Z and $-Z$, and average the pair. Their errors partly cancel.
$\hat{C}_{\text{cv}}, \beta$	Control variate: subtract the known error of something correlated. S_T has known mean $S_0 e^{rT}$, so if the sample's $\bar{S}_T$ came out high, the call estimate probably did too. β sets how much to correct.
importance sampling	Draw Z from a shifted bell curve so paths land near the strike, then multiply each payoff by a likelihood-ratio weight so the average stays unbiased. It is Girsanov's theorem (section 6.2) used as a tool.

In plain words: the error of Monte Carlo depends on the spread of what you average. Each trick shrinks that spread without changing the mean. Antithetic pairs cancel noise when the payoff moves steadily with Z . A control variate removes the part of the noise you can explain with something whose answer you already know. Importance sampling stops wasting paths where nothing happens. Which one wins depends on the payoff: near the money, the control variate is best, and far out of the money, importance sampling is. At a strike of 150, antithetic pairs don't help at all, because almost every pair pays zero both ways.



Horizontal axis: payoff evaluations (log scale). Every method gets the same budget of 20,000.

Strike 100: Black–Scholes price 9.4134. With 20,000 payoff evaluations each: plain 9.3824 ± 0.1010 ; antithetic 9.4542 ± 0.0756 ($1.8\times$ less variance); control variate 9.4254 ± 0.0417 ($5.9\times$ less variance); importance sampling 9.3824 ± 0.1010 ($1.0\times$ less variance).

Variance reduction is free accuracy: a $5\times$ variance reduction is worth $5\times$ more paths. No method wins everywhere. Pick the one that matches where the payoff's information is. In practice these tricks are combined, and quasi-random numbers (sequences spread out more evenly than random ones) add a further gain for smooth payoffs.

Going deeper: the precise statement

Control variates. For Y with $\mathbb{E}[Y]$ unknown and X with $\mathbb{E}[X]$ known, $Y - \beta(X - \mathbb{E}X)$ is unbiased for every β , and its variance $\text{Var } Y - 2\beta \text{Cov}(X, Y) + \beta^2 \text{Var } X$ is smallest at $\beta^* = \text{Cov}(X, Y) / \text{Var } X$. There it equals $(1 - \rho_{XY}^2) \text{Var } Y$, so the reduction factor is $1/(1 - \rho^2)$. Estimating β from the same sample adds a bias of order $1/n$, which is negligible.

Antithetic variates. $\text{Var} \left(\frac{1}{2}(f(Z) + f(-Z)) \right) = \frac{1}{2} \text{Var } f(Z) (1 + \text{Corr}(f(Z), f(-Z)))$. It helps exactly when $f(Z)$ and $f(-Z)$ are negatively correlated, which is guaranteed for monotone f .

Importance sampling. If $Z \sim N(0, 1)$ is replaced by $Z + \theta$, then $\mathbb{E}[f(Z)] = \mathbb{E}[f(Z + \theta) e^{-\theta(Z+\theta)+\theta^2/2}]$. The weight is the Radon–Nikodym derivative of section 6.2. A good θ puts the sampling mass where $|f| \times \text{density}$ is large, and a bad one can increase the variance.

Show the code (Python + NumPy)

```
import numpy as np
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S0, K, sig, r, T, n = 100.0, 150.0, 0.2, 0.03, 1.0, 20_000 # a deep out-of-the-money call
d1 = (log(S0 / K) + (r + sig ** 2 / 2) * T) / (sig * sqrt(T))
exact = S0 * N(d1) - K * exp(-r * T) * N(d1 - sig * sqrt(T))
Z = np.random.default_rng(25).normal(size=n)
ST = lambda z: S0 * np.exp((r - sig ** 2 / 2) * T + sig * sqrt(T) * z)
pay = lambda z: exp(-r * T) * np.maximum(ST(z) - K, 0)
plain = pay(Z)
anti = 0.5 * (pay(Z[: n // 2]) + pay(-Z[: n // 2])) # same 20,000 payoff
evaluations
beta = np.cov(plain, ST(Z))[0, 1] / ST(Z).var()
control = plain - beta * (ST(Z) - S0 * exp(r * T)) # E[S_T] = S0 e^{rT} is known
theta = (log(K / S0) - (r - sig ** 2 / 2) * T) / (sig * sqrt(T))
importance = pay(Z + theta) * np.exp(-theta * (Z + theta) + theta ** 2 / 2) # Girsanov
weight
print(f"Black-Scholes {exact:.4f}")
for name, y in [("plain", plain), ("antithetic", anti), ("control", control),
("importance", importance)]:
    print(f"{name:11} {y.mean():.4f} +- {y.std() / sqrt(len(y)):.4f}")
```

Output when we ran it:

```
Black-Scholes 0.2818
plain          0.2885 +- 0.0176
antithetic     0.3057 +- 0.0182
control        0.2913 +- 0.0162
importance     0.2835 +- 0.0023
```

9.2 Greeks by Simulation

Traders need sensitivities as much as prices: delta ($\partial V / \partial S$) sets the hedge (section 6.1), and gamma ($\partial^2 V / \partial S^2$) says how quickly the hedge goes stale. The obvious way is to reprice with the stock bumped up and down and divide by the bump. It works, but only if done carefully, and there are two cleverer ways that need no bump at all.

Predict first: Estimate delta by bumping the stock by a tiny h , repricing each side with its own independent random numbers, and dividing by $2h$. The estimate will be...

Very accurate

Extremely noisy

Biased but stable

Answer: "Extremely noisy". With independent random numbers, each price has noise of about ± 0.1 , and dividing that difference by a tiny $2h$ blows the noise up. Common random numbers, or the pathwise and likelihood-ratio methods, avoid this.

Pick delta or gamma and drag the bump size h . Each row is one estimator from the same 20,000 paths, drawn as a dot with a ± 2 standard error bar. The dashed line is the exact Black–Scholes value.

$$\Delta_{\text{pathwise}} = \mathbb{E} \left[e^{-rT} \mathbf{1}_{\{S_T > K\}} \frac{S_T}{S_0} \right], \quad \Delta_{\text{LR}} = \mathbb{E} \left[e^{-rT} (S_T - K)^+ \frac{Z}{S_0 \sigma \sqrt{T}} \right]$$

Δ Delta: how much the option's value changes when the stock moves up by one dollar.

h The bump size in the finite-difference estimate $(V(S_0 + h) - V(S_0 - h)) / 2h$.

common random numbers Reprice both bumped values with the *same* random draws, so their noise cancels in the difference.

$\mathbf{1}_{\{S_T > K\}}$ 1 if the option finishes in the money, 0 otherwise.

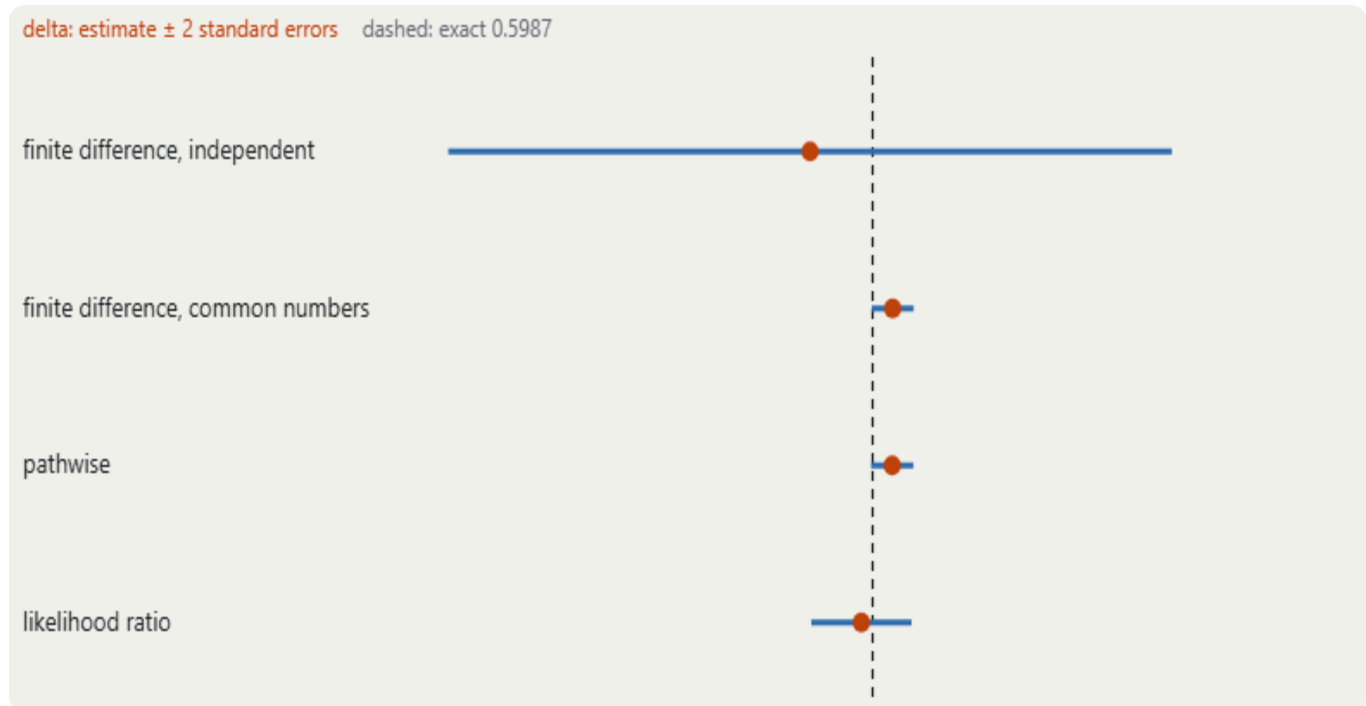
pathwise Differentiate each path's payoff with respect to S_0 and average. Here $\partial S_T / \partial S_0 = S_T / S_0$.

Z The standard normal number that generated the path: $S_T = S_0 e^{(r - \sigma^2/2)T + \sigma\sqrt{T}Z}$.

likelihood ratio (LR) Leave the payoff alone and differentiate the *probability* of the path instead. The weight $Z / (S_0 \sigma \sqrt{T})$ is the derivative of the log density.

In plain words: finite differences are a trade-off. A big bump is biased, because the curve isn't straight over it. A small bump divides noise by a small number. Common random numbers remove most of that noise, because both repricings see the same paths. The pathwise method is the most precise but needs a payoff you can differentiate: it works for delta, but a call's payoff

has a kink at the strike, so it can't give gamma. The likelihood-ratio method never differentiates the payoff, so it works even for kinked or jumping payoffs such as digitals. It pays for that with more noise. Gamma for a call is exactly the case where finite differences struggle and the likelihood ratio shines.



Bump $h = 1.00$. finite difference, independent: 0.5743 ± 0.0706 ; finite difference, common numbers: 0.6065 ± 0.0041 ; pathwise: 0.6064 ± 0.0041 ; likelihood ratio: 0.5943 ± 0.0098 . Exact delta: 0.5987.

For delta, the pathwise estimator is the most precise and needs no bump at all. For gamma of a call, finite differences with a small bump blow up even with common random numbers, while the likelihood ratio stays unbiased. Real risk systems use exactly these ideas, plus automatic differentiation, which computes pathwise Greeks for every input in one extra pass.

Going deeper: the precise statement

Pathwise method. If the payoff f is Lipschitz continuous (a call is, a digital is not), then $\frac{\partial}{\partial S_0} \mathbb{E}[f(S_T)] = \mathbb{E}[f'(S_T) \partial S_T / \partial S_0]$: the derivative can be moved inside the expectation.

Likelihood ratio. Write the price as $\int f(s) g(s; S_0) ds$ with g the density of S_T . Then $\partial_{S_0} \text{price} = \mathbb{E}[f(S_T) \partial_{S_0} \log g(S_T; S_0)]$, which works for any integrable f . For GBM the score is $Z / (S_0 \sigma \sqrt{T})$. Gamma uses the second derivative: $((Z^2 - 1) / (S_0^2 \sigma^2 T) - Z / (S_0^2 \sigma \sqrt{T}))$.

Finite differences. The central difference has bias $O(h^2)$. With independent samples its variance is $O(1/(nh^2))$ for delta and $O(1/(nh^4))$ for gamma. Common random numbers reduce this to $O(1/n)$ for smooth payoffs. For gamma of a kinked payoff it only improves to $O(1/(nh))$.

Show the code (Python + NumPy)

```

import numpy as np
from math import erf, exp, log, sqrt, pi
S0, K, sig, r, T, n = 100.0, 100.0, 0.2, 0.03, 1.0, 100_000
d1 = (log(S0 / K) + (r + sig ** 2 / 2) * T) / (sig * sqrt(T))
delta, gamma = 0.5 * (1 + erf(d1 / sqrt(2))), exp(-d1 ** 2 / 2) / sqrt(2 * pi) / (S0 * sig
* sqrt(T))
Z = np.random.default_rng(26).normal(size=n)
ST = S0 * np.exp((r - sig ** 2 / 2) * T + sig * sqrt(T) * Z)
payoff = exp(-r * T) * np.maximum(ST - K, 0)
pathwise = exp(-r * T) * (ST > K) * ST / S0 # d(payoff)/dS0 along each
path
lr_delta = payoff * Z / (S0 * sig * sqrt(T)) # payoff x d(log
density)/dS0
lr_gamma = payoff * ((Z ** 2 - 1) / (S0 ** 2 * sig ** 2 * T) - Z / (S0 ** 2 * sig *
sqrt(T)))
se = lambda y: y.std() / sqrt(len(y))
print(f"delta: exact {delta:.4f} pathwise {pathwise.mean():.4f} +- {se(pathwise):.4f} LR
{lr_delta.mean():.4f} +- {se(lr_delta):.4f}")
print(f"gamma: exact {gamma:.5f} LR {lr_gamma.mean():.5f} +- {se(lr_gamma):.5f}")

```

Output when we ran it:

```

delta: exact 0.5987 pathwise 0.5991 +- 0.0019 LR 0.5964 +- 0.0045
gamma: exact 0.01933 LR 0.01920 +- 0.00043

```

9.3 Barrier Options

A *down-and-out call* is a call that dies if the stock ever touches a barrier below it: here the stock and strike are at 100 and the barrier at 90. Section 2.3's reflection principle gives an exact price when the barrier is watched continuously. A simulation can only check the barrier at discrete dates, and that creates a systematic error.

Predict first: If the simulation checks the barrier only once a day, its price for the knock-out option will be...

Too high

Too low

Exact, up to random noise

Answer: "Too high". Between two checks a path can dip below the barrier and come back up unseen. Those paths should have died but are counted as alive, so the simulated price is too high: about 3% with daily checks and 16% with quarterly ones.

Choose how often the barrier is checked. On the left, knocked-out paths turn grey. On the right, orange shows plain Monte Carlo and blue the corrected version, across all checking frequencies, against the exact price (dashed).

$$P(\text{path dips below } B \text{ between checks} \mid x_i, x_{i+1}) = \exp\left(-\frac{2(x_i - \ln B)(x_{i+1} - \ln B)}{\sigma^2 \Delta t}\right)$$



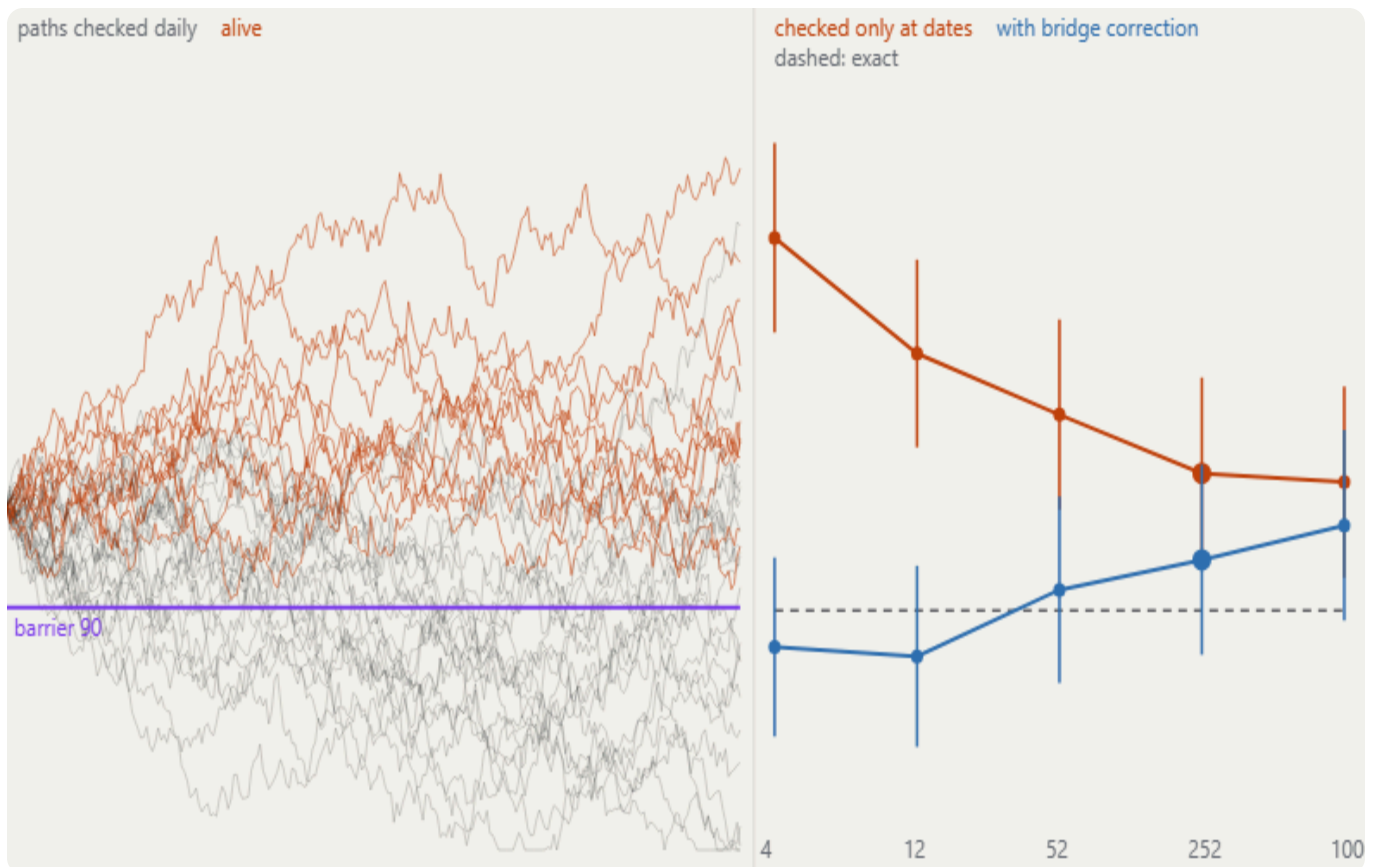
B The barrier: 90. If the stock touches it at any moment, the option is worthless.

x_i, x_{i+1} The log stock price at two consecutive checking dates, both above $\ln B$.

Δt The time between checks.

Brownian bridge A Brownian path pinned at both ends (section 1.1's bridge construction). Its chance of dipping below a level has the exact formula above.

In plain words: the stock moves continuously, but the simulation only looks at it now and then. Between looks a path can cross the barrier and come back, and the simulation never notices. It overprices the knock-out, by +16% with quarterly checks, +11% monthly, +6% weekly and +3% daily, and the error shrinks only like $\sqrt{\Delta t}$. The fix costs almost nothing: between each pair of checks, ask how likely a Brownian bridge with those endpoints was to touch the barrier (the formula above, the reflection principle again), and weight the path by its chance of surviving. That removes the bias at every frequency.



Exact price (barrier watched continuously): 7.740. Checked daily: plain Monte Carlo 8.149 ± 0.143 , with the Brownian-bridge correction 7.891 ± 0.141 .

Discrete monitoring overprices knock-out options, and the error fades only slowly as checks get more frequent. The Brownian-bridge correction fixes it exactly for GBM. The same idea gives the Broadie–Glasserman–Kou rule: a discretely monitored barrier behaves like a continuous one shifted by a factor $e^{\pm 0.5826 \sigma \sqrt{\Delta t}}$. It matters for real contracts, which specify daily closing prices.

Going deeper: the precise statement

Closed form (continuous monitoring, $B \leq K$). With $\lambda = (r + \sigma^2/2)/\sigma^2$ and $y = \frac{\ln(B^2/(S_0 K))}{\sigma \sqrt{T}} + \lambda \sigma \sqrt{T}$, the down-and-in call is $C_{\text{di}} = S_0 (B/S_0)^{2\lambda} N(y) - K e^{-rT} (B/S_0)^{2\lambda-2} N(y - \sigma \sqrt{T})$, and down-and-out = vanilla – down-and-in. The derivation reflects paths at their first touch of B (section 2.3), after changing measure to remove the drift (section 6.2).

Bridge crossing probability. For Brownian motion with variance σ^2 per unit time, conditioned on $X_0 = a > b$ and $X_{\Delta t} = c > b$, the probability of touching b is $\exp(-2(a-b)(c-b)/(\sigma^2 \Delta t))$. The drift drops out of the conditioned path.

Continuity correction. Broadie, Glasserman & Kou showed that a barrier checked every Δt prices like a continuously monitored one moved away from the stock by the factor $e^{\beta\sigma\sqrt{\Delta t}}$, with $\beta = -\zeta(1/2)/\sqrt{2\pi} \approx 0.5826$.

References: M. Broadie, P. Glasserman & S. Kou, "A continuity correction for discrete barrier options", *Mathematical Finance* 7 (1997), 325–349. Glasserman, *Monte Carlo Methods in Financial Engineering*, ch. 6. Shreve, *Stochastic Calculus for Finance II*, §7.3.

Show the code (Python + NumPy)

```
import numpy as np
from math import erf, exp, log, sqrt
N = lambda x: 0.5 * (1 + erf(x / sqrt(2)))
S0, K, B, sig, r, T = 100.0, 100.0, 90.0, 0.2, 0.03, 1.0
v, lam = sig * sqrt(T), (r + sig ** 2 / 2) / sig ** 2
d1 = (log(S0 / K) + (r + sig ** 2 / 2) * T) / v
call = S0 * N(d1) - K * exp(-r * T) * N(d1 - v)
y = log(B ** 2 / (S0 * K)) / v + lam * v
exact = call - (S0 * (B / S0) ** (2 * lam) * N(y) - K * exp(-r * T) * (B / S0) ** (2 * lam - 2) * N(y - v))
rng, steps, chunks = np.random.default_rng(27), 252, 5 # checked daily; 5 x 20,000
paths
dt = T / steps
plain, fixed = [], []
for _ in range(chunks):
    x = np.log(S0) + np.cumsum((r - sig ** 2 / 2) * dt + sig * sqrt(dt) * rng.normal(size=(20_000, steps))), axis=1)
    x = np.hstack([np.full((20_000, 1), log(S0)), x]) - log(B) # distance above the
    barrier, in log terms
    payoff = exp(-r * T) * np.maximum(np.exp(x[:, -1]) * B - K, 0) * (x > 0).all(axis=1)
    bridge = np.prod(1 - np.exp(-2 * np.clip(x[:, :-1], 0, None) * np.clip(x[:, 1:], 0,
    None) / (sig ** 2 * dt)), axis=1)
    plain.append(payoff); fixed.append(payoff * bridge)
for name, p in [("daily checks only", np.concatenate(plain)), ("with bridge fix",
np.concatenate(fixed))]:
    print(f"{name:17} {p.mean():.4f} +- {p.std() / sqrt(len(p)):.4f} (exact
{exact:.4f})")
```

Output when we ran it:

```
daily checks only 8.0062 +- 0.0443 (exact 7.7402)
with bridge fix 7.7793 +- 0.0439 (exact 7.7402)
```

9.4 American Options

An *American* put can be exercised at any moment before maturity, not just at the end. Pricing it means solving an *optimal stopping* problem: at every moment, is it better to take $K - S$ now or keep the option alive? Trees solve this easily in one dimension. The Longstaff–Schwartz method solves it with simulation, which also scales to many assets.

Predict first: With a positive interest rate, how does the American put compare with the otherwise identical European put?

Worth exactly the same

Worth strictly more

Worth less, because exercising early wastes time value

Answer: "Worth strictly more". Exercising early locks in $K - S$ now, and the cash earns interest. When the stock has fallen far enough, that beats waiting. So the American put is worth strictly more than the European one when $r > 0$: here 6.09 against 5.57.

Grey paths end where they are exercised (orange dots). The black curve is the exact exercise boundary from a binomial tree, and the blue dots are the boundary Longstaff–Schwartz discovers. Change the number of paths and the regression basis.

$$V_0 = \sup_{\tau \leq T} \mathbb{E} \left[e^{-r\tau} (K - S_\tau)^+ \right], \quad \text{exercise at } t_j \text{ if } (K - S_{t_j})^+ > \hat{\mathbb{E}}[\text{discounted future cash flows} \mid \mathcal{F}_{t_j}]$$



τ A *stopping time*: an exercise rule that decides using only the prices seen so far.

$\sup_{\tau}$ The best possible value over all such rules.

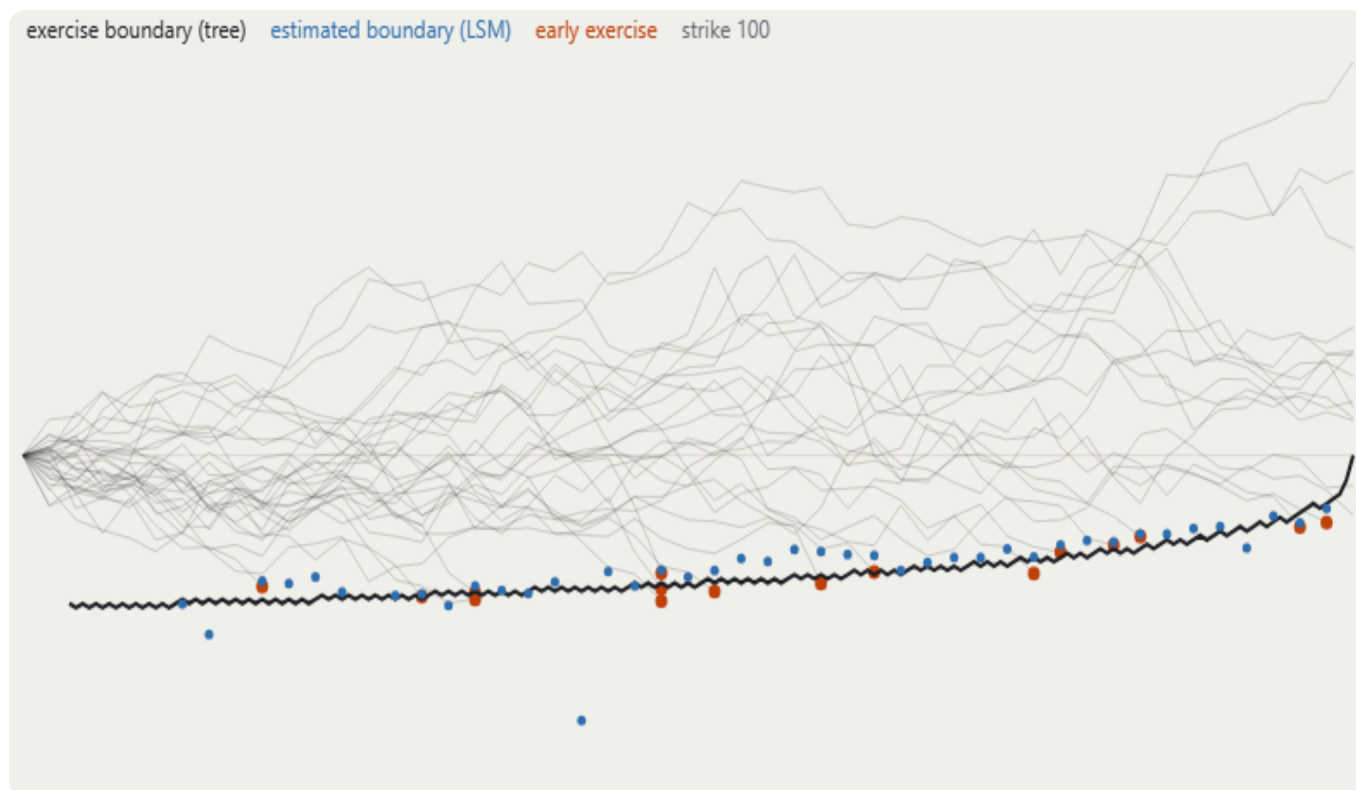
t_j The exercise dates in the simulation: 50 dates over the year.

$\hat{\mathbb{E}}[\cdot \mid \mathcal{F}_{t_j}]$ The *continuation value*: what holding on is worth, estimated by a regression of the discounted future cash flows on $1, S, S^2, \dots$ across the simulated paths.

exercise boundary The critical stock price below which exercising immediately is optimal. It rises toward K as maturity nears.

In plain words: work backwards from maturity. At the last date the choice is obvious. At each earlier date, you need to know what waiting is worth, which depends on the unknown future. Longstaff and Schwartz's idea is to estimate it by regression across paths: among paths with a

similar stock price today, the average discounted payoff they went on to receive is the value of waiting. Exercise wherever the immediate payoff beats that estimate. The rule it learns traces out the exercise boundary. Because the rule is only an estimate of the best rule, the price is typically slightly low, so a stronger regression basis or more paths moves it up toward the tree's value.



Longstaff–Schwartz with 10,000 paths and basis S, S^2, S^3 : 6.132 ± 0.072 . Binomial tree (1000 steps): 6.090. European put (no early exercise): 5.574, so the right to exercise early is worth 0.516.

The right to exercise early is worth about 0.52 here, 8.5% of the option. Longstaff–Schwartz recovers both the price and the exercise boundary from simulation alone, which is why it is the standard tool for American and Bermudan options on several assets, where trees and PDE grids become impractical. An American *call* on a stock without dividends is never exercised early: it is worth more alive (Merton's result).

Going deeper: the precise statement

Optimal stopping. The price is the Snell envelope $V_t = \text{ess sup}_{\tau \geq t} \mathbb{E}[e^{-r(\tau-t)}(K - S_\tau)^+ | \mathcal{F}_t]$, the smallest supermartingale above the payoff. In PDE form it is a free-boundary problem: $\partial_t V + \frac{1}{2}\sigma^2 S^2 \partial_{SS} V + rS \partial_S V - rV = 0$ above a boundary $S^*(t)$, with $V = K - S$ below it and V and $\partial_S V$ continuous across it ("smooth pasting").

Bias. Any fixed exercise rule applied to fresh paths gives a *lower* bound, since no rule beats the optimal one. Duality (Rogers; Haugh & Kogan) gives matching upper bounds. Estimating the rule and

the price on the same paths can bias the estimate slightly upward, so production code uses separate paths for the two steps.

Calls. Without dividends, $C_{Am} \geq S - Ke^{-r(T-t)} > S - K$ before maturity, so exercising a call early is never optimal (Merton, 1973).

References: F. Longstaff & E. Schwartz, "Valuing American options by simulation: a simple least-squares approach", *Review of Financial Studies* 14 (2001), 113–147. Shreve, *Stochastic Calculus for Finance II*, ch. 8. Glasserman, *Monte Carlo Methods in Financial Engineering*, ch. 8.

Show the code (Python + NumPy)

```
import numpy as np
from math import exp, sqrt
S0, K, r, sig, T, dates, n = 100.0, 100.0, 0.05, 0.2, 1.0, 50, 50_000
h = T / dates
rng = np.random.default_rng(28)
S = S0 * np.exp(np.cumsum((r - sig ** 2 / 2) * h + sig * sqrt(h) * rng.normal(size=(n,
dates))), axis=1))
cash = np.maximum(K - S[:, -1], 0.0) # value if held to the end
for j in range(dates - 2, -1, -1): # walk backwards through
    exercise dates
    cash *= exp(-r * h)
    itm = S[:, j] < K
    X = np.vander(S[itm, j] / S0, 4) # basis 1, S, S^2, S^3
    continuation = X @ np.linalg.lstsq(X, cash[itm], rcond=None)[0]
    exercise = K - S[itm, j] > continuation
    cash[np.flatnonzero(itm)[exercise]] = (K - S[itm, j])[exercise]
price = exp(-r * h) * cash
print(f"Longstaff-Schwartz American put: {price.mean():.3f} +- {price.std() / sqrt(n):.3f}
(binomial tree: 6.090)")
```

Output when we ran it:

```
Longstaff-Schwartz American put: 6.056 +- 0.032 (binomial tree: 6.090)
```

Chapter recap

Quick questions on this chapter's big ideas. Each answer is explained as soon as you choose.

1. Plain Monte Carlo with n paths has standard error proportional to...

$1/n$

$1/\sqrt{n}$

$1/n^2$

Answer: " $1/\sqrt{n}$ ". The standard error is $sd/\sqrt{n}$: 100 times more paths buy only 10 times more accuracy, which is why variance reduction matters.

2. Which Greek estimator works even for a payoff with a jump, such as a digital option?

Pathwise

Likelihood ratio

Neither

Answer: "Likelihood ratio". The likelihood-ratio method differentiates the probability density, not the payoff, so it needs no smoothness. The pathwise method fails at jumps.

3. Checking a knock-out barrier only daily in a simulation makes the option's price...

Too high

Too low

Exact

Answer: "Too high". Crossings between checks are missed, so paths that should die survive. The Brownian-bridge correction removes the bias.